

Compile Pokemon Yellow From Source

CS 407 Intro To Linux

April 27, 2026

1 Introduction

We've taken a peek at programming in a few languages such as C, Python, bash and Rust. Hopefully you've seen that Linux is a wonderful OS for doing software development! Today we'll do a fun activity! We'll configure a tool chain for compiling assembly language for the gameboy, and then we'll compile Pokemon Yellow! We'll verify the checksum of the ROM we compile to make sure it's legit, and then we'll run it on an emulator on the internet! Then, if you want, you can install an emulator on your phone and play your game there.

Our goals are:

1. Compile a valid pokemon yellow ROM that is the same as the official Nintendo ROM. The sha1sum will be `cc7d03262ebfaf2f06772c1a480c7d9d5f4a38e1`
2. Then we'll hack the code a bit and you'll make a custom ROM with whatever changes you want to make.

2 Why Gameboy?

Why not? It's cool. I personally love Gameboy because it's one of the last platforms where you can play a game offline. No updates, no servers, no nonsense! Just you and a game.

3 Resources

We're going to use

1. The pokemon source code.

2. The RGBDS toolchain

This worksheet will guide you through the relevant parts of these references. Ready? Let's do it!

4 Install dependencies

Let's install some stuff we'll need later.

Listing 1: Install this stuff

```
1 root@machine$ apt install gcc binutils git make
```

5 Set up RGBDS

I'll be guiding you through instructions from the resources above. If any command I show you here doesn't work, then consult the resources above and let me know so I can update this guide.

5.1 Install rgbds

RGBDS has the assembler, linker, etc. needed to turn the source code into a ROM! First thing you need to do is get the .tar.xz file onto your droplet. You can copy the link from the RGBDS website as shown in figure 1.

Then, use `wget` to download the .tar.xz onto your droplet as shown in figure 2.

After you've done that we can make a directory called **rgdbs** and extract the .tar.xz into the directory called **rgbds**, as shown in figure 3.

Then, we're finally ready to install our rgbds toolchain! Enter the **rgbds** directory and run **./install.sh**. This bash script will take care of the installation. Use **less** to look at the install script if you want! You should see some familiar stuff in there! Have a look at figure 4 to see the commands you need to type.

5.2 Verify RGBDS is installed

As mentioned above, you should run **rgbasm -V** (capital V) to verify that the assembler was installed.

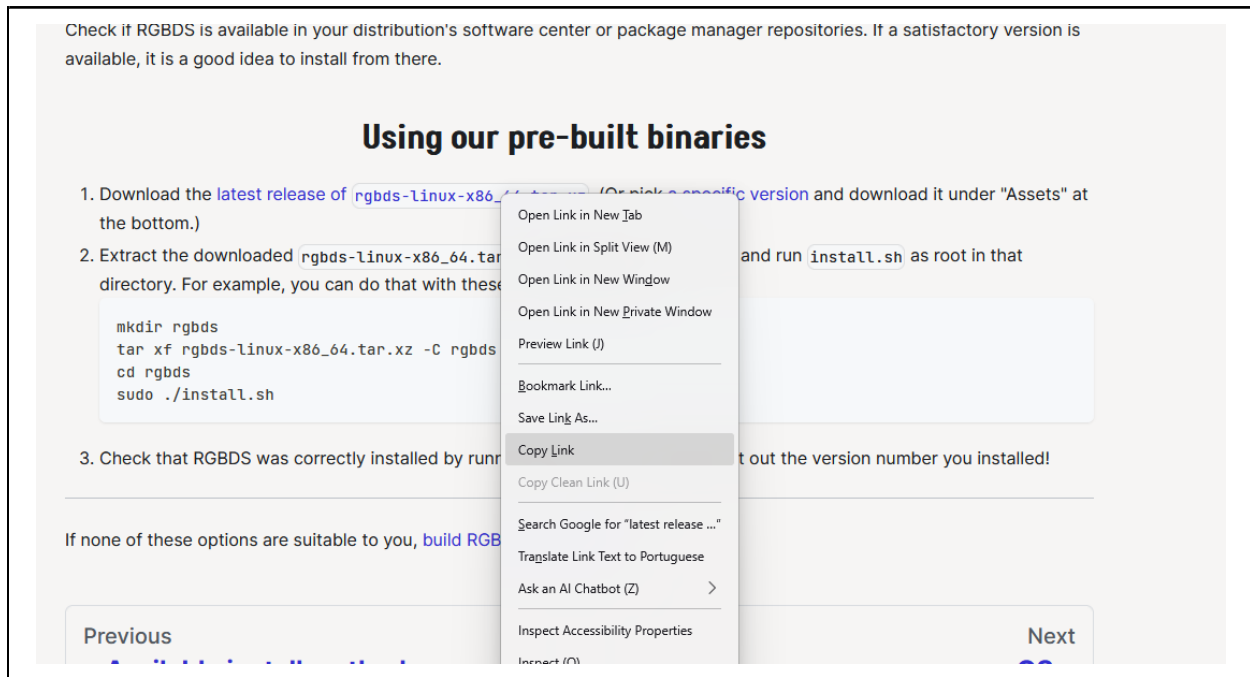


Figure 1: Right click on the link to the .tar.xz and copy the link.



Figure 2: Use `wget` to download the .tar.xz. Then use `ls` to see that it downloaded.

```
root@pokemon:~# ls
rgbds-linux-x86_64.tar.xz
root@pokemon:~# mkdir rgbds
root@pokemon:~# tar -xvf rgbds-linux-x86_64.tar.xz -C rgbds/
rgbasm
rgblink
rgbfix
rgbgfx
gbz80.7
rgbasm-old.5
rgbasm.1
rgbasm.5
rgbds.5
rgbds.7
rgbfix.1
rgbgfx.1
rgblink.1
rgblink.5
install.sh
```

Figure 3: `mkdir` and `tar -xvf`. Be sure to use the `-C` flag to make sure we extract to the directory we made!

```
root@pokemon:~# cd rgbds/
root@pokemon:~/rgbds# ./install.sh
root@pokemon:~/rgbds# rgbasm -V
rgbasm v1.0.1
root@pokemon:~/rgbds# ls /usr/local/bin/
rgbasm  rgbfix  rgbgfx  rgblink
root@pokemon:~/rgbds#
```

Figure 4: Follow along with me here to install. After you install, you can run those commands I listed to verify that `rgbasm` is installed. Then you can look in `/usr/local/bin` where you'll see all the stuff we installed.

6 Download and Build Pokemon Yellow!

Now we're ready to download and compile pokemon yellow.

First you'll need to clone the code from github. Go to the repo on github:

`https://github.com/pret/pokeyellow`

and then click the green button to get the link to HTTPS remote (not the SSH one!), as shown in figure 5. After you've done this, clone it to the root user's HOME directory on your droplet, as shown in figure 6. Then you can enter the repo and build it!

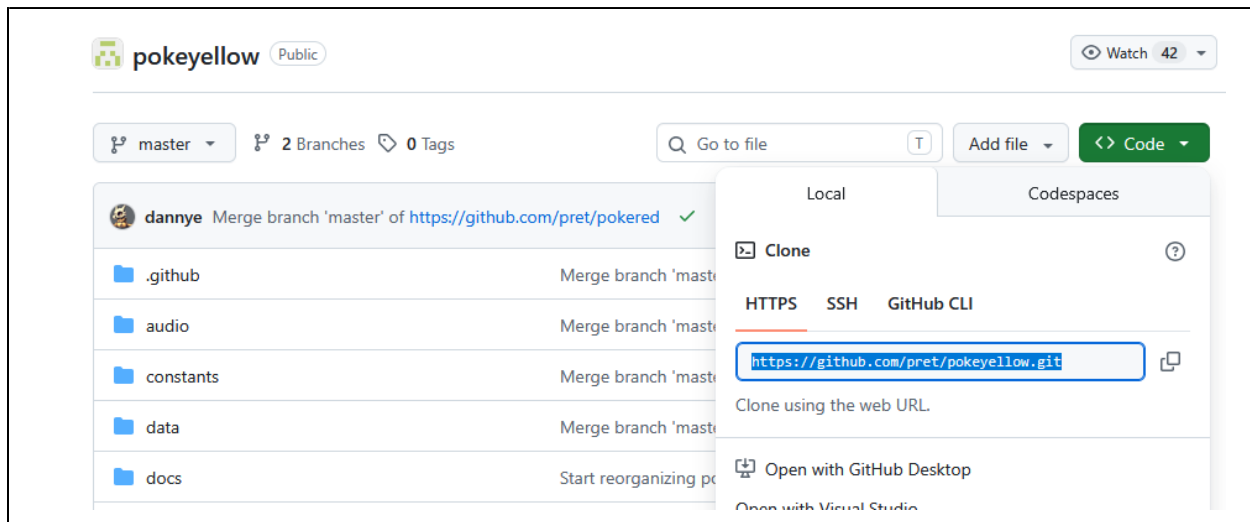


Figure 5: Get the https remote.

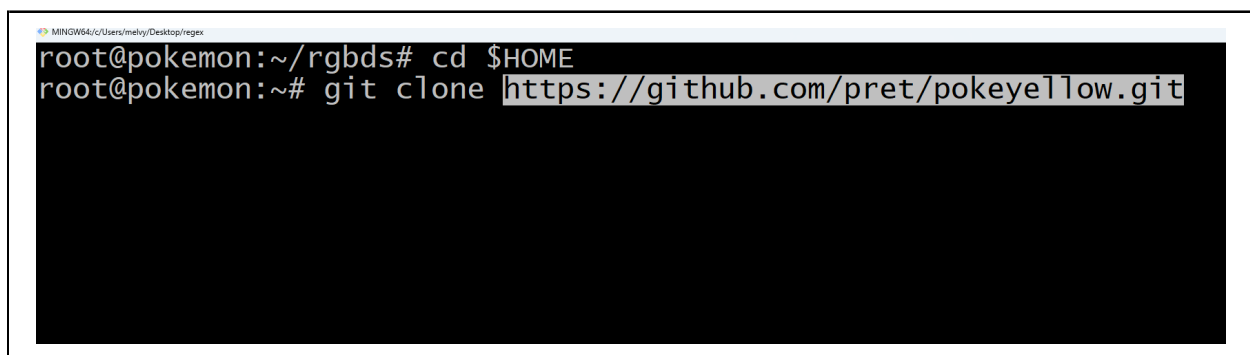


Figure 6: Clone the repository.

As we've seen in the past when we compiled *onetrueawk* from source, we often use the **make** build system on Linux to build our projects. Take a moment to look at the **Makefile** in

pokeyellow. Run **less Makefile** and scroll through there. In this class we don't have time to talk about Makefiles, but they are incredibly important. Type **q** to exit **less** when you're done reading.

When you're ready, go to the directory where the **Makefile** is and type **make**! Look at figure 7. After it builds you should have a **pokeyellow.gbc** and the shalsum should be correct! See figure 8. Congratulations! You have an official copy of pokemon yellow!!

```
root@pokemon:~# cd pokeyellow/
root@pokemon:~/pokeyellow# ls
INSTALL.md  constants  gfx          macros      ram.asm      text.asm
Makefile    data       home         main.asm    rgbdscheck.asm tools
README.md   docs       home.asm     maps        roms.sha1    vc
audio       engine     includes.asm maps.asm     scripts
audio.asm   garbage    layout.link  ram         text
root@pokemon:~/pokeyellow# less Makefile
root@pokemon:~/pokeyellow# make
```

Figure 7: Build the ROM. Be sure to hit [ENTER] after you type make. I didn't hit [ENTER] because then the screen would fill with build output.

```
root@pokemon:~/pokeyellow# ls
INSTALL.md  gfx          maps.asm      ram_debug.o
Makefile    home         maps.o        rgbdscheck.asm
README.md   home.asm     maps_debug.o  rgbdscheck.o
audio       home.o       pokeyellow.gbc roms.sha1
audio.asm   home_debug.o pokeyellow.map scripts
audio.o     includes.asm pokeyellow.sym text
audio_debug.o layout.link  pokeyellow_debug.gbc text.asm
constants  macros       pokeyellow_debug.map text.o
data        main.asm     pokeyellow_debug.sym text_debug.o
docs        main.o       ram           tools
engine      main_debug.o ram.asm       vc
garbage     maps        ram.o
root@pokemon:~/pokeyellow# sha1sum pokeyellow.gbc
cc7d03262ebfaf2f06772c1a480c7d9d5f4a38e1 pokeyellow.gbc
root@pokemon:~/pokeyellow#
```

Figure 8: Make sure the checksum is correct.

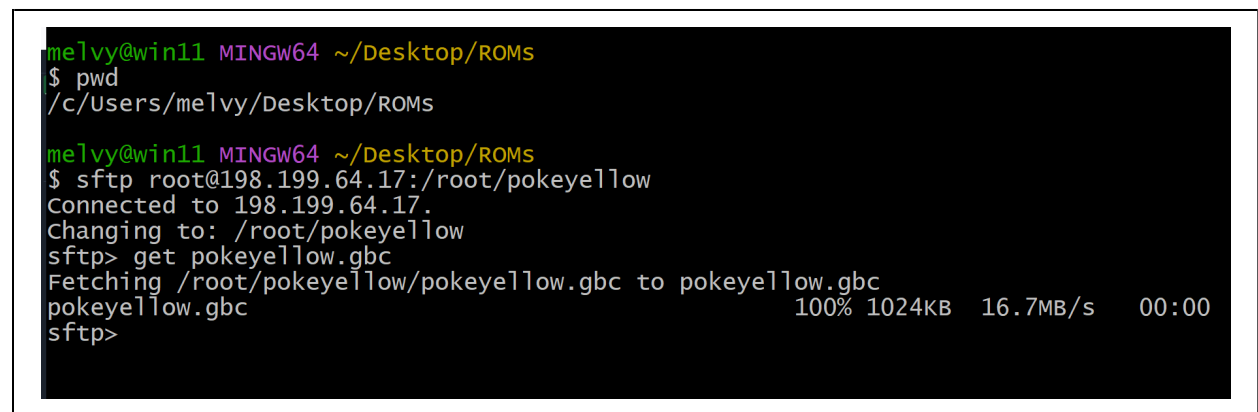
7 Play Your Game

Now you can play your game! Use **sftp** to copy the ROM off of your droplet onto your PC. See figure 9 to see how I download the ROM with **sftp**. Then go to

<https://gba.nicholas-vancise.dev/>

and upload your ROM. Then you can play it!! Have a look at figure 10 to see me playing it on my computer using the browser emulator.

The source code for this gbajs3 emulator is freely available on github, alongside the code for the rgbds toolchain and the pokemon yellow source code.

A terminal window with a black background and green text. The prompt is 'melvy@win11 MINGW64 ~/Desktop/ROMs'. The user enters '\$ pwd' and the output is '/c/Users/melvy/Desktop/ROMs'. Then the user enters '\$ sftp root@198.199.64.17:/root/pokeyellow'. The output shows 'Connected to 198.199.64.17.', 'Changing to: /root/pokeyellow', 'sftp> get pokeyellow.gbc', 'Fetching /root/pokeyellow/pokeyellow.gbc to pokeyellow.gbc', 'pokeyellow.gbc 100% 1024KB 16.7MB/s 00:00', and 'sftp>'.

```
melvy@win11 MINGW64 ~/Desktop/ROMs
$ pwd
/c/Users/melvy/Desktop/ROMs

melvy@win11 MINGW64 ~/Desktop/ROMs
$ sftp root@198.199.64.17:/root/pokeyellow
Connected to 198.199.64.17.
Changing to: /root/pokeyellow
sftp> get pokeyellow.gbc
Fetching /root/pokeyellow/pokeyellow.gbc to pokeyellow.gbc
pokeyellow.gbc 100% 1024KB 16.7MB/s 00:00
sftp>
```

Figure 9: Remember our old friend SFTP? Pay attention to the directories!

7.1 Other good emulators

On my Linux PC I use **mgba** (specifically I **apt install mgba-qt** and use that frontend). On my android phone I use **Pizzaboy pro**. On iOS I use **delta**. I can't say these emulators are safe or good, they're just what I use and have had a great experience with.

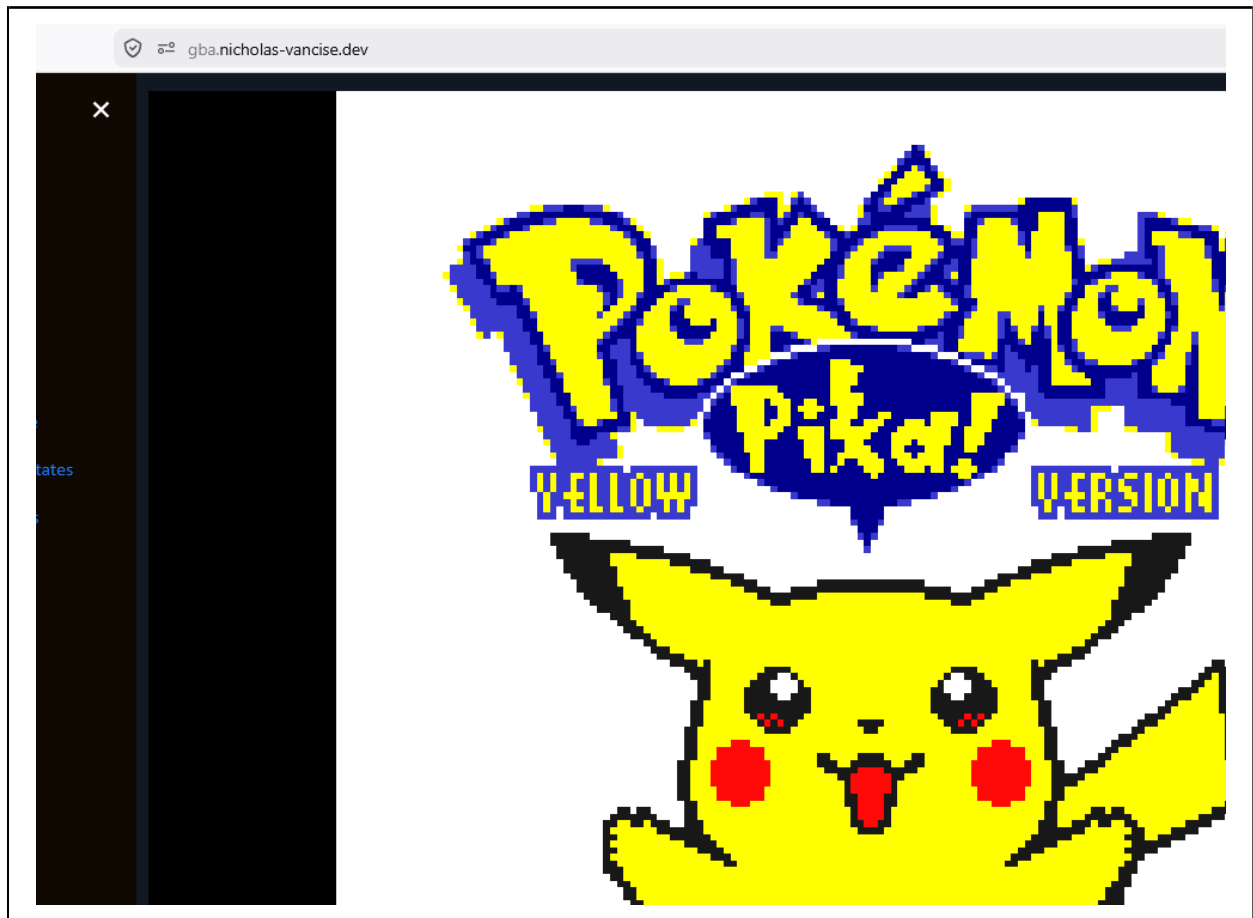


Figure 10: It works!

8 Hack it!

8.1 Important Note About The Emulator!

This emulator appears to aggressively save state in the browser. It does autosaves. To get around that, when I want to upload a new ROM (for example, your hack) I need to clear all the old state. Before you try to run your hacked rom, try this:

1. Refresh the page. This will stop the old ROM.
2. Then click **File System** on the side, and go through and delete everything that's there. See figure 11

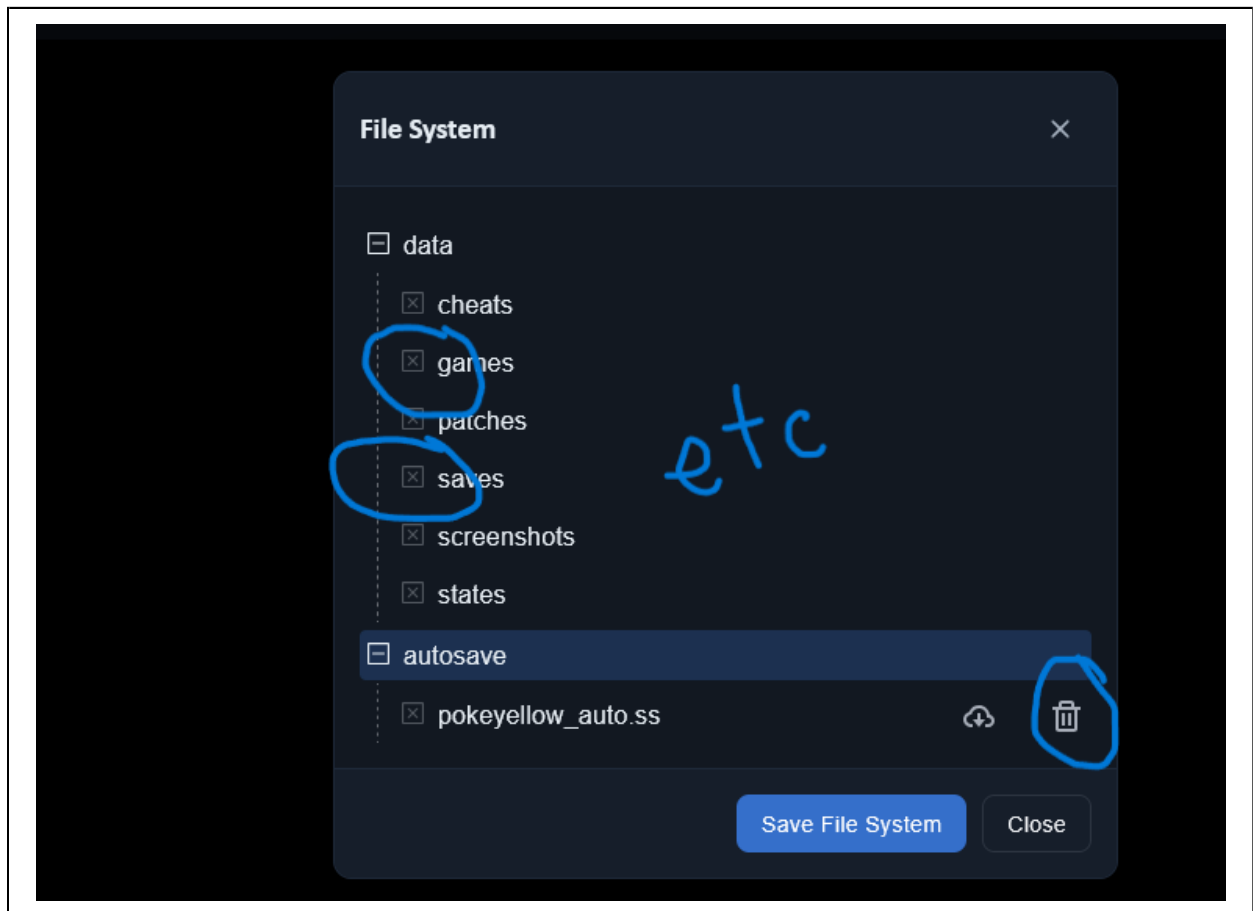


Figure 11: Delete everything from the filesystem.

Okay, now you're ready to do the hack.

8.2 Hack #1 - Change some text

You can change the text in the game as a first easy quick hack. Have a look at figure 12 and see that, when the game starts, Professor Oak introduces himself. With our knowledge of `grep` and `sed`, we can find the location in the code where this text is, and change it! Have a look at figure 13



Figure 12: Hmmm maybe we can change this...

After you make your changes, you might want to look at them to make sure you didn't do anything wrong. You can use the `git status` command to see what files you changed and use the `git diff` command to see what changes you made. Have a look at figures 14 and 15.

If you're happy with your changes, make sure you're in the `pokeyellow` directory and run `make` again. This will rebuild the code. The test out the new `pokeyellow.gbc` ROM in the emulator! NOTE! The `sha1sum` for the ROM changed! It is no longer the original checksum, indicating that it's a different ROM. Cool, huh? Have a look at figure 16 to see

```

root@pokemon:~/pokeyellow# grep -iR "my name is" data/
data/text/text_3.asm:  para "My name is OAK!"
root@pokemon:~/pokeyellow# sed -i 's/My name is OAK!/My name is MEL!/g' data/text/text_3.asm
root@pokemon:~/pokeyellow# grep -iR "my name is" data/
data/text/text_3.asm:  para "My name is MEL!"
root@pokemon:~/pokeyellow#

```

Figure 13: I think I WILL BE THE PROFESSOR!

```

root@pokemon:~/pokeyellow# git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working c
      modified:   data/text/text_3.asm

no changes added to commit (use "git add" and/or "git commit -
root@pokemon:~/pokeyellow#

```

Figure 14: What the status of the repo?

it rebuild and have a look at figure 17 to see it in action on the emulator.

```

root@pokemon:~/pokeyellow# git diff
diff --git a/data/text/text_3.asm b/data/text/text_3.asm
index a4a3c883..d82c7607 100644
--- a/data/text/text_3.asm
+++ b/data/text/text_3.asm
@@ -437,7 +437,7 @@ _oakSpeechText1::
     line "welcome to the"
     cont "world of #MON!"

-    para "My name is OAK!"
+    para "My name is MEL!"
     line "People call me"
     cont "the #MON PROF!"
     prompt

```

Figure 15: What changes have I made?

```

root@pokemon:~/pokeyellow# make
rgbasm -weverything -wtruncation=1 -Q8 -P includes.asm -o te
rgblink -weverything -wtruncation=1 -d -p 0x00 -l layout.lin
eyellow.sym -o pokeyellow.gbc audio.o home.o main.o maps.o ra
/pikachu.o gfx/sprites.o gfx/surfing_pikachu.o gfx/tilesets.o
rgbfix -weverything -cjsv -k 01 -l 0x33 -m MBC5+RAM+BATTERY -
-p 0x00 pokeyellow.gbc
rgbasm -weverything -wtruncation=1 -Q8 -P includes.asm -D _D
asm
rgblink -weverything -wtruncation=1 -d -p 0xff -l layout.lin
-n pokeyellow_debug.sym -o pokeyellow_debug.gbc audio_debug.o
o maps_debug.o ram_debug.o text_debug.o gfx/pics_debug.o gfx/
es_debug.o gfx/surfing_pikachu_debug.o gfx/tilesets_debug.o
rgbfix -weverything -cjsv -k 01 -l 0x33 -m MBC5+RAM+BATTERY -
-p 0xff pokeyellow_debug.gbc
root@pokemon:~/pokeyellow# sha1sum pokeyellow.gbc
7210a6a71c2b68577408bd15e78284fb797d8fcc pokeyellow.gbc
root@pokemon:~/pokeyellow#

```

Figure 16: Run `make` again and then check the `sha1sum`



Figure 17: hacked!

8.3 Hack #2 - Change the pokemon that follows you

First, be sure to clear the emulator state before you do this hack, otherwise the emulator might keep trying to run the old ROM with the old save state.

The classic cool thing about Pokemon Yellow is that Pikachu follows you around. We can change it, however, so that it's not pikachu that follows you!

There are many sprites in the **gfx/sprites** directory. It would be easiest for you to see all of the sprites if you look on github:

<https://github.com/pret/pokeyellow/tree/master/gfx/sprites>

You cannot use any random image for this hack!!! Be sure to use the **monster.png** image as shown in figure 18. Follow along with the steps in figure 18 to change the pikachu sprite to the monster sprite. The rebuild the game and play your ROM for a few minutes until you receive pikachu from professor oak. Then you will see that pikachu is not following you, but a monster is!

```
root@pokemon:~# cd pokeyellow/gfx/sprites/
root@pokemon:~# cd pokeyellow/gfx/sprites/
root@pokemon:~/pokeyellow/gfx/sprites# cp monster.png pikachu.png
root@pokemon:~/pokeyellow/gfx/sprites# cd ../../
root@pokemon:~/pokeyellow# git status
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   data/text/text_3.asm
        modified:   gfx/sprites/pikachu.png

no changes added to commit (use "git add" and/or "git commit -a")
root@pokemon:~/pokeyellow# make
```

Figure 18: Change Pikachu

TODO: Improve this section. Sorry, ran out of time, didn't have time take a bunch of screenshots!

TODO: Provide students with a save state that has pikachu following them so they dont have to play the game up until the time when you have a pikachu. This probably takes 5-10 minutes in game to get to this point.

9 Conclusions

Linux is an amazing platform for any sort of software project you want to do! Hope this lab was an exciting and eye-opening experience. You've applied your knowledge of Linux and the bash shell to do a really interesting project that you likely would not have been able to do a few months ago!

All of the stuff we've done today was done as hobby projects by talented and inspired people. Think about what had to happen to make this lab possible:

- *pret* on github had to decompile the pokemon game into assembly
- People had to figure out how gameboy worked and design the assembler **rgbasm**
- People like endrift had to design the mGBA emulator
- People like Nick Van Cise had to get a website working and get the mGBA emulator in the browser!
- etc. etc. etc. So much amazing work by so many amazing people.

9.1 Want More?

Check out the *pret* account on github. There's source code for many pokemon games. I'm currently playing Emerald. Also check out this amazing book that was just published this year about Gameboy Assembly Programming.